

Тема 4. Система от типове данни – единични и комбинирани. S-изрази. Функции от първи клас. Анонимни функции

1. Система от типове данни – единични и комбинирани

Elixir използва динамична и силно типизирана система от данни, наследена от Erlang. Всички стойности са *immutable* (непроменяеми) – веднъж създадени, те не могат да бъдат променени, а се създава нова стойност.

Таблица 1.

Data type	Usage	Description
Numeric	<code>x = 21</code>	Elixir поддържа не само цели числа, но и числа с плаваща запетая. Числата могат да се дефинират също така в осмична, шестнадесетична и двоична система.
Atom	<code>:true</code>	Атомите са константи, чието име е същото като тяхната стойност
Boolean	<code>: false, :true</code>	true или false , обикновено се декларират като атоми
Strings	<code>"Hello"</code>	Низовете са затворени в двойни кавички (<code>" "</code>), а многоредовите низове са затворени в тройни двойни кавички (<code>""" """</code>)
Lists	<code>['a', 10, :true]</code>	Списъците се използват за съхраняване на различни типове стойности и са представени в квадратни скоби <code>[]</code>
Tuples	<code>{'apple', 100, :false}</code>	Кортежите са подобни на списъците и са представени във къдрави скоби <code>{}</code> . Кортежите са подходящи за достъп до стойностите, а списъците за вмъкване и изтриване на стойности

Типовете могат да се разделят на две групи

Единични (primitive)

- числа
- boolean
- atom
- string

Комбинирани (composite)

- list
- tuple
- map
- struct

1.1. Числов тип (Number)

Elixir има два основни числови типа:

- Integer
- Float

Elixir, както всеки друг език за програмиране, поддържа както цели числа, така и числа с плаваща запетая.

Примери

```
1 # В десетична бройна система (БС)
#=> 1

10 000
#=> 10 000
```

```
0x53 # В шестнадесетична БС
#=> 83
```

```
0o53 # В осмична БС
#=> 43
```

```
0b11 # В двоична БС
#=> 3
```

Float изискват точка, след която се записва поне една цифра.

```
iex> 1.0  
1.0  
  
3.14  
#=> 3.14
```

Числа с плаваща запетая

Този тип също поддържа е (експонента) за число на степен

Пример

```
iex> 1.0e-10  
1.0e-10  
  
1.0e-10  
#=> 1.0e-10
```

Числа с плаваща запетая в експоненциален вид

10 145 230 000 може да се запише, като 1.014 523 e10

Float в Elixir са със 64-битова двойна точност.

Функцията round връща най-близкото цяло число към даден float.

Функцията trunc връща целочислената част на даден float.

Пример

```
iex> round 3.58 # Резултатът е: 4  
iex> trunc 3.58 # Резултатът е: 3
```

Могат да се дефинират и числа с двоична, осмична, шестнадесетична основа. Elixir поддържа бързи нотации за въвеждане на двоични, осмични и шестнадесетични числа.

● Двоично число

Дефиниране на число с двоична основа става, като се добави пред него '0b'.

Пример: 0b1101 е двоично число, чийто еквивалент е 13 в десетична бройна система.

● Осмично число

Дефиниране на число с осмична основа става, като се добави пред него '0o'.

Пример: 0o52 е осмично число, чийто еквивалент е 42 с десетична бройна система.

● Шестнадесетично число

Дефиниране на число със шестнадесетична основа става, като се добави пред него '0x'.

Примери

```
iex> 0b1010  
10  
iex> 0o777  
511  
iex> 0x1F  
31
```

Пример

```
a = 10  
b = 3.14  
IO.puts(a + b)
```

Изход

```
13.14
```

Поддържат се стандартни оператори:

- + - * /
- div

- rem

Примери

```
iex> 1 + 2          Резултатът е: 3
iex> 5 * 5          Резултатът е: 25
```

Ако се въведе:

```
iex> 10 / 2
5.0
```

резултатът е 10/2 и връща float 5.0, вместо integer 5. Това е така, защото в Elixir операторът „/“ винаги връща float.

Функцията div връща цялата част от делението.

Функцията rem връща остатъка от делението.

```
iex> div(10, 2)
5
iex> div 10, 2
5
iex> rem 10, 3
1
```

Важно е да се знае, че скобите не са необходими при извикване на функция.

Пример

```
div(10,3)
rem(10,3)
```

Изход

```
3
1
```

1.2. Boolean тип

Boolean стойности са:

- true
- false

Elixir поддържа true и false като булеви променливи

```
iex> true
true

iex> true == false
false
```

Пример

```
a = 10 > 5
IO.puts(a)
```

Изход

```
true
```

Логически оператори:

```
and
or
not
```

Пример

```
true and false
```

Elixir предоставя куп предикатни функции за проверка на типа стойност.

Например: функцията is_boolean/1 може да се използва за проверка дали дадена стойност е булева или не.

Забележка: Функциите в Elixir се идентифицират по име и по брой аргументи (т.е. arity). Следователно `is_boolean/1` идентифицира функция с име `is_boolean`, която приема 1 аргумент. `is_boolean/2` идентифицира различна функция със същото име, но различно arity (т.е. брой аргументи).

```
iex> is_boolean(true)
true
iex> is_boolean(1)
false
```

Можем също да използваме `is_integer/1`, `is_float/1` или `is_number/1`, за да провери съответно дали един аргумент е цяло число, плаващо число или едно от двете.

Забележка: След въвеждане на `h` в shell-а се осигурява достъп до документацията към всяка функция.

Например: въвеждането на `h is_integer/1` ще отпечата документацията за функцията `is_integer/1`. Помощникът работи и с оператори и други конструкции (може да се въведе `h ==/2`).

1.3. Atom

Atom е константа, чиято стойност е нейното име.

Пример

```
:ok
:error
:hello
```

Atoms често се използват за:

- статуси
- идентификатори
- ключове в `map`

Пример

```
status = :ok
```

Atoms са глобално уникални и се съхраняват в таблица.

В Elixir (и в Erlang, върху който се базира), атомът е основен тип данни, който представлява името на нещо, подобно на символ или константа. Той има фиксирана идентичност и често се използва за идентификатори, ключове или състояния.

Основни характеристики на атомите:

- Не се променят – атомът е уникален и едентичността му не зависи от стойността му.
- Съхраняват се в системната таблица на BEAM – всяко уникално име се запамятава веднъж.
- Сравняват се бързо – тъй като са уникални идентификатори.
- Не са числа или низове, въпреки че могат да приличат на тях.

Синтаксис

- **Прост атом** – започва с малка буква и без кавички:

```
:ok
:error
:foo
```

- **Атом с кавички** – може да съдържа специални символи или интервали:

```
:"Hello World"
:"123"
:"my atom!"
```

Примери за използване

```
# Функция, която връща статус
def check(value) do
  if value > 0 do
```

```
:ok
else
:error
end
end

check(10) #=> :ok
check(-1) #=> :error
```

Използване на атоми като ключове на Map

```
map = %{name: "Alice", status: :active}
map[:status] #=> :active
```

Атомите са константи, а не стойности.

- Използват се за комуникация между процеси, pattern matching, ключове на map и status кодове.
- В BEAM виртуалната машина са много ефективни за сравнение и съхранение.

Малка аналогия: атомът е като именован ключ – уникално име, което не се променя, а не контейнер за данни.

Атомите са константи, чието име е тяхната собствена стойност. Някои програмни езици ги наричат още символи:

Пример:

```
iex> :hello
:hello

iex> :hello == :world
false
```

Булевите true и false също са атоми:

Пример:

```
iex> true == :true
true

iex> is_atom(false)
true

iex> is_boolean(:false)
true
```

1.4. String

String е последователност от UTF-8 символи.

Пример

```
name = "Elixir"
IO.puts(name)
```

Низове в Elixir се вмъкват между двойни кавички и са кодирани в UTF-8. Те могат да обхванат няколко реда и да съдържат интерполации.

Едноредов низ се дефинира чрез двойни кавички:

```
"Hello world,,"
```

Многоредов низ се дефинира чрез тройни двойни кавички (синтаксисът е подобен на Python)

```
"""
Hello
World!
"""
```

Пример за дефиниране на едноредов коментар с участие на специфични символи като „ö“:

```
iex> "hellö"
```

```
"hellö,,
```

Забележка: ако работим под Windows, възможно е терминала да не използва UTF-8 по подразбиране. Тогава е необходимо да се промени кодирането на текущата сесия, като се стартира „chcp 65001“.

Интерполация на низ се поддържа в Elixir:

```
iex> "hellö #{:world}"  
"hellö world"
```

Низовете могат да имат прекъсвания на редове в тях или да ги въвеждат, като се използват бягащи последователности.

Пример

```
iex> "hello  
...> world"  
"hello\nworld"  
  
iex> "hello\nworld"  
"hello\nworld,,
```

Отпечатване на низ чрез функцията IO.puts/1 от IO модула:

```
iex> IO.puts "hello\nworld"  
hello  
world  
:ok
```

Забележете, че функцията IO.puts/1 връща атома: ok, като резултат след отпечатване. Низовете в Elixir са представени вътрешно от двоични файлове, които са поредица от байтове:

```
iex> is_binary("hellö")  
true
```

Връщане на броя байтове от string:

```
iex> byte_size("hellö")  
6
```

Забележете, че броят байтове в този string е 6, въпреки че има 5 знака. Това е така, защото символът „ö“ отнема 2 байта, за да бъде представен в UTF-8. Можем да получим действителната дължина на низ, въз основа на броя знаци, като използваме функцията String.length/1:

```
iex> String.length("hellö")  
5
```

Модулът String съдържа куп функции, които работят с низове, както е дефинирано в стандарта Unicode:

```
iex> String.upcase("hellö")  
"HELLÖ"
```

Операция конкатенация:

```
"Hello " <> "World"
```

Изход

```
Hello World
```

Двоични файлове

Двоичните файлове са поредици от байтове, затворени в <<>>, разделени със запетая.

Пример

```
<< 65, 68, 75>>
```

Двоичните файлове се използват предимно за обработка на данни, свързани с битове и байтове, ако се разполага с такива. Те могат по подразбиране да съхраняват от 0 до 255 бита за всяка стойност. Този размер може да бъде увеличен, като се използва функцията за размер, която посочва колко бита трябва да са необходими за съхраняване на определена стойност.

Пример

```
<<65, 255, 289::size(15)>>
```

1.5. (Linked) List

List е свързан списък.

За да се укаже list със стойности, се използват квадратни скоби, като стойностите могат да бъдат от всякакъв тип.

Типа на елементите може да е различен в списъка:

```
iex> [1, 2, true, 3]
```

```
[1, 2, true, 3]
```

```
[1, "Hello", :an_atom, true]
```

Присвояване на променливи в списък

```
list = [1,2,3,4] (присвояване)
```

```
iex> list = [1,2,3] (присвояване)
```

Основни операции - извличане на head и tail.

Те могат да бъдат извлечени с функциите hd/1 и tl/1. Т.е. при работа с list в Elixir има вградени функции за head (е първият елемент от списъка) и tail на list-a (останалата част от списъка) с имена hd и tl, които връщат съответно главата и опашката на списъка.

```
head | tail
```

Пример

```
iex> hd(list) (извличане на head)
```

```
1
```

```
iex> tl(list) (извличане на tail)
```

```
[2, 3]
```

Пример

```
[h | t] = [1,2,3]
```

```
IO.inspect(h)
```

```
IO.inspect(t)
```

Изход

```
1
```

```
[2,3]
```

Дължината на списък се извлича чрез length

```
iex> length [1, 2, 3]
```

```
3
```

Два списъка могат да бъдат **конкатенирани** и **извадени** с помощта на операторите ++/2 и -/2:

```
iex> [1, 2, 3] ++ [4, 5, 6]
```

```
[1, 2, 3, 4, 5, 6]
```

```
iex> [1, true, 2, false, 3, true] -- [true, false]
```

```
[1, 2, 3, true] (т.е. true и false са премахнати)
```

Понякога, когато се създава списък, той връща списък със символи. Това е така, защото когато Elixir види списък с ASCII знаци за печат, той го отпечата като списък със знаци. Трябва да се знае, че, низовете и списъците със знаци не са равни.

head или tail на empty list (празен списък), при опит за извличане от празен списък водят до изхвърляне на грешка:

```
iex> hd []  
** (ArgumentError) argument error
```

Понякога при създаване на списък връща стойност в единични кавички:

```
iex> [11, 12, 13]  
"\v\fr"  
  
iex> [104, 101, 108, 108, 111]  
'hello'
```

Когато Elixir види списък с ASCII номера за печат, той го отпечата като списък със символи (буквално списък от знаци). Списъците със символи са доста често срещани при взаимодействие със съществуващия код на Erlang. Трябва да се има предвид, че еднотитираните и двойните цитати не са еквивалентни в Elixir, тъй като са представени от различни типове:

```
iex> 'hello' == "hello"  
false
```

Единичните кавички са списъци със знаци, двойните кавички са низове.

1.6. Tuple

Tuple е фиксирана колекция от елементи. Дефиниране на кортеж в Elixir става чрез къдрави скоби. Подобно на списъците, кортежите могат да съдържат всякакви стойности:

Пример

```
iex> {:ok, "hello"}  
{:ok, "hello"}
```

Кортежите съхраняват елементи последователно в паметта. Това означава, че достъпът до елемент на кортеж чрез индекс или получаването на размера на кортежа е бърза операция (индексите започват от нула).

Достъп до елемент

```
t = {:ok, 200}  
elem(t,1)
```

Изход

```
200
```

```
iex> tuple = {:ok, "hello"}  
{:ok, "hello"}  
  
iex> elem(tuple, 1) – това ще изведе 1-вият елемент от кортежа, а той е hello, нулевият е :ok  
"hello,,  
  
iex> tuple_size(tuple) – кортежа има 2 елемента  
2
```

Задаване на елемент в определен индекс в кортеж с put_elem/3:

```
iex> tuple = {:ok, "hello"}  
{:ok, "hello"}  
  
iex> put_elem(tuple, 1, "world")  
{:ok, "world"}
```



```
iex> tuple  
{:ok, "hello"}
```

Забележете, че **put_elem/3** връща нов кортеж. Оригиналният кортеж, съхраняван в променливата `tuple`, не е променен, тъй като типовете данни на Elixir са неизменни. Неизменният код на Elixir е по-лесен за разсъждение, тъй като никога не трябва да се променя. Това е полезно при мутиране на код, тъй като не се променя структурата от данни на това място. Това елиминира често срещани случаи, при които има паралелен код и има състезателни условия, при които два различни обекта се опитват да променят структурата на данните едновременно.

1.7. List или Tuple?

Кога се използва?

List - колекции с много елементи

Tuple - фиксирани данни

Примери

с List

```
[1,2,3,4]
```

с Tuple

```
{:ok, result}
```

Tuple се използва често за резултати от функции.

Защо са необходими различни колекции - списъци и кортежи, когато работят по един и същи начин?

Защото те имат различна реализация. **Списъците** се съхраняват като **свързани списъци**, така че вмъкване и изтриване на елементи са бързи операции **при списъка**. **Кортежите** пък се съхраняват в **непрекъснат блок памет**, което прави достъпа до тях по-бърз, но забавя операциите вмъкване и изтриване.

Каква е всъщност разликата между списъци и кортежи?

Списък

Структурата данни "Свързан списък" представлява поредица от елементи, като всеки от тях съдържа не само информацията, която трябва да се пази в масив, но и информацията относно това кой е следващият елемент в поредицата. В стандартната имплементация, тази информация представлява указател към следващия елемент. Така лесно може да се обхождат всички елементи от структурата, като се започне от първия, следван от неговия указател към следващия, а от там към по-следващия и т.н. Допълнително се запазват двата указателя един към първия и един към последния елемент на списъка. Това позволява лесно да се добави елемент, както в началото, така и в края на списъка. Свързаният списък се имплементира в езици, като C# чрез класа `LinkedList<T>`. За разлика от C#, в Elixir списъците се съхраняват в паметта, като свързани списъци, което означава, че всеки елемент в списъка има своята стойност и сочи към следващия елемент, докато се достигне края на списъка. Ние наричаме всяка двойка стойност и указател:

```
iex> list = [1|[2|[3|[]]]]  
[1, 2, 3]
```

Кортежите се съхраняват непрекъснато в паметта. Това означава, че достъпът до дължината на списък е линейна операция, като е необходимо да се преминава през целия списък, за да се разбере неговия размер. Т.е. има бързо получаване на **размера на кортежа или достъп до елемент по индекс**. Актуализирането или добавянето на елементи към кортежите е "скъпа" операция, защото изисква копиране на целия кортеж в паметта. Това определя операцията добавяне на елемент към списък, като бърза операция, стига да има готови елементи:

Примери

```
1) iex> [0] ++ list
```

```
[0, 1, 2, 3]
```

```
2) iex> list ++ [4]
```

```
[1, 2, 3, 4] # на базата на първоначалния пример
```

Изводите от двата примера са: **При 1)**, първата операция е бърза, защото просто се добавя нова стойност, която сочи към останалият списък, а **при 2)**, добавяне (актуализиране) на елемент е бавна операция, защото трябва да се възстанови (копира) целия списък в паметта и да се добави нов елемент в края.

Един много често срещан случай на използване на кортежи е връщане на допълнителна информация от функция. Например: `File.read/1` е функция, която може да се използва за четене на съдържанието на файла и връща кортежи:

```
iex> File.read("path/to/existing/file")
{:ok, "... contents ..."}

iex> File.read("path/to/unknown/file")
{:error, :enoent}
```

Ако пътят, даден на `File.read/1`, съществува, той връща кортеж **c atom: ok** като първи елемент и **съдържанието на файла** като втори. В противен случай връща кортеж **c: error** и **описанието на грешката**. Съществува функция `elem/2` за достъп до кортеж, но няма вграден еквивалент за списъци:

```
iex> tuple = {:ok, "hello"}
{:ok, "hello"}

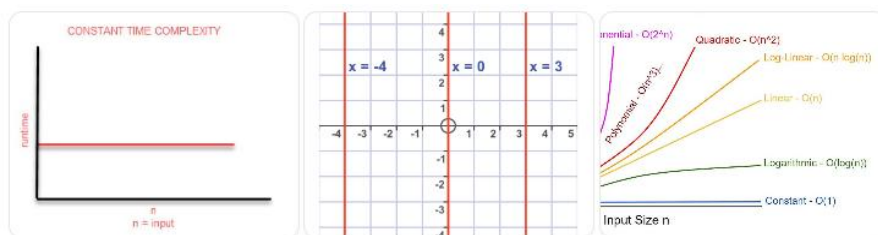
iex> elem(tuple, 1)
"hello"
```

При изброяване на елементите в структура от данни, Elixir следва практически правило за именуване на функции:

- използват се функции като `byte_size/1` и `tuple_size/1`, когато операцията е **константна по време ($O(1)$)**, т.е. размерът е предварително известен и не изисква обхождане на структурата;
- използват се функции като `length/1` и `String.length/1`, когато операцията изисква обхождане (**$O(n)$**) на структурата, за да се преброят елементите.

$O(1)$ и **$O(n)$** са означения от теорията за алгоритмите, наречена **Big-O** нотация (асимптотична сложност). Те показват колко бързо расте времето за изпълнение (или паметта), когато входът става по-голям.

$O(1)$ – константна сложност

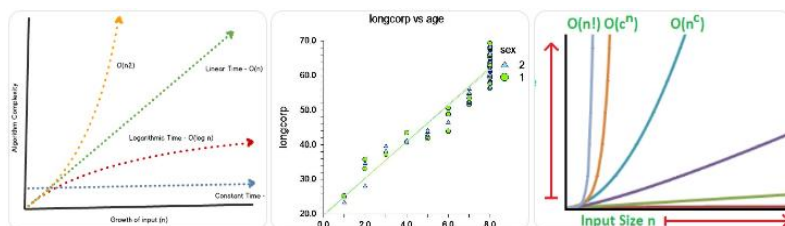


Означава, че операцията отнема еднакво време, независимо от размера на данните.

Примери в Elixir:

```
tuple_size({1,2,3})
byte_size("hello")
```

Без значение дали имаш 5 или 5 милиона елемента — времето е почти едно и също.



Означава, че времето расте пропорционално на броя елементи (n).

Примери в Elixir:

```
length([1,2,3,4,5])  
String.length("hello")
```

Ако данните са 10 пъти повече - операцията ще отнеме ~10 пъти повече време.

- $O(1)$ - „С един поглед се знае отговора“
- $O(n)$ - „Трябва да се премине през всичко и да се преброи“
- $\text{byte_size}/1 - O(1)$ (знае се размера веднага)
- $\text{length}/1 - O(n)$ (обхожда списъка)
- $\text{String.length}/1 - O(n)$ (чете целия низ за Unicode символи)

Например:

- $\text{byte_size}/1$ връща броя байтове в бинарен низ (string в Elixir е бинарен тип);
- $\text{tuple_size}/1$ връща броя на елементите в кортежа;
- $\text{length}/1$ връща дължината на списък (list), като го обхожда;
- $\text{String.length}/1$ връща броя на Unicode символите (graphemes) в низ.

Извод:

- $\text{byte_size}/1$ е „евтина“ операция ($O(1)$), защото размерът на бинарния низ е известен;
- $\text{String.length}/1$ е по-скъпа ($O(n)$), защото трябва да се обходи целия низ, за да преброи Unicode символите, които могат да заемат повече от един байт.

Elixir също така предоставя Port, Reference и PID като типове данни (обикновено се използват в комуникацията на процеса). Elixir предоставя следните аритметични оператори +, -, *, /, както и функциите div/2 и rem/2 за целочислено деление и остатък.

Elixir предлага за обединение на два списъци ++ и премахване на елементи от списъка --:

```
iex> [1,2,3] ++ [4,5,6]  
[1,2,3,4,5,6]  
iex> [1,2,3] -- [2]  
[1,3]
```

Свързването на низове се извършва с <>:

```
iex> "foo" <> "bar"  
"foobar"
```

Elixir предоставя и три логически оператора: or, and и not. Тези оператори са строги в смисъл, че очакват булева стойност (true или false) като първи аргумент:

```
iex> true and true  
true  
iex> false or is_atom(:example)  
true
```

Предоставянето на булева стойност ще предизвика изключение:

```
iex> 1 and true  
** (ArgumentError) argument error
```

or и and са оператори на “късо съединение”. Те изпълняват дясната страна само, ако лявата страна не е достатъчна за определяне на резултата:

```
iex> false and error("This error will never be raised")  
false  
  
iex> true or error("This error will never be raised")  
true
```

Освен тези логически оператори, Elixir предоставя също оператор за логическо или, и, и различно т.е. ||, && и !, които приемат аргументи от всякакъв тип. За тези оператори всички стойности с изключение на false и nil ще се оценят на true:

Примери

```
# or
iex> 1 || true
1

iex> false || 11
11

# and
iex> nil && 13
nil

iex> true && 17
17

# !
iex> !true
false

iex> !1
false

iex> !nil
true
```

Ако някой от аргументите не е булев, се използва **&&**, **||** и **!**.

Elixir също така предоставя и оператори за сравнение: **==**, **!=**, **===**, **!==**, **<=**, **>=**, **<** и **>**:

Примери

```
iex> 1 == 1
true

iex> 1 != 2
true

iex> 1 < 2
true
```

Разликата между **==** и **===** е, че последната е по-строга при сравняване на integer и float:

```
iex> 1 == 1.0
true

iex> 1 === 1.0
false
```

В Elixir можем да сравним също така и два различни типа данни:

```
iex> 1 < :atom
true
```

Причината, поради която можем да сравним различни типове данни, е **прагматизмът**.

Общият ред за сортиране е дефиниран по следния начин:

```
number < atom < reference < functions < port < pid < tuple < maps < list < bitstring
```

2. S-изрази

Elixir кодът вътрешно се представя като S-expression подобна структура се нарича AST (Abstract Syntax Tree). S-изразът (sexpr или sexp) е начин да се представи вложен списък от данни. Той означава "символично изразяване" и обикновено се среща в езика за програмиране Lisp и вариантите на му, като Scheme, Racket и Clojure. Тези езици за програмиране използват S-изрази за представяне на компютърната програма, както и за данните на програмата. За да се представи математически израз "Пет пъти сумата от седем и три" той се записва като S-израз с префиксна нотация. В Lisp S-изразът може да се запише по следния начин:

```
(* 5 (+ 7 3))
```

S-изразите намират приложение и в Elixir. Метапрограмирането е важна функция в Elixir. То дава възможност за писане на код, който, при компилиране се трансформира в различна на себе си версия. Този вид код се нарича макрос или това е „код, който пише код“. За да работи, макросът трябва да има достъп до вътрешното представяне на езика на изходния код. Това обикновено е дървовидна структура от данни, наречена Abstract Syntax Tree (Абстрактно синтаксично дърво). Изразът $1 + 2 * 4$ може да се представи концептуално чрез AST:

```
+  
/\   
1  *  
  /\   
  2  4
```

а резулата е 9.

В много програмни езици представянето на кода чрез абстрактно синтаксично дърво (AST) е скрито от програмиста, докато Elixir го прави достъпно и позволява неговата директна манипулация чрез макроси. Макросите получават AST, трансформират го чрез добавяне, премахване или анализ на елементи и генерират нов код, като редица основни конструкции в езика (напр. `def`, `defmodule`) са реализирани именно по този начин. Освен това Elixir предоставя инструменти за парсане и обхождане на AST, което позволява както създаване на макроси, така и извършване на статичен анализ на кода.

2.1. AST

AST е структурирано представяне на кода. За демонстрация на AST ще се използва макроса `quote/2`. В Elixir `quote/2` е специална функция (по-точно макро), която се използва за превръщане на Elixir код в AST (Abstract Syntax Tree), без той да бъде изпълнен. Взима код и го „цитира“ като данни (AST), вместо да го изпълни.

Пример:

```
quote do  
  1 + 2  
end
```

Резултат:

```
{:+, [context: Elixir, import: Kernel], [1, 2]}
```

Това е вътрешното AST представяне на израза $1 + 2$.

- `:+` е операторът/функцията, която се извиква (в случая операторът `+` е представен като атом `:+`)
- `[context: Elixir, import: Kernel]` са метаданни (metadata) за израза, като
 - ✓ `context: Elixir` - контекстът, в който е създаден
 - ✓ `import: Kernel` - операторът идва от стандартния модул `Kernel`
- `[1, 2]` това са аргументите на функцията/оператора

Цялата структура следва шаблона:

```
{име на функция, метаданни, аргументи}
```

Като се извиква функцията с аргументите `[1, 2]`

`quote` е основен инструмент при макросите, защото:

- позволява да работиш с код като данни
- можеш да го анализираш или променяш
- след това да върнеш нов код

`quote/2` означава, че макросът приема 2 аргумента:

- опции (по избор)
- блок с код (`do ... end`)

Пример с опции:

```
quote location: :keep do  
  x + y  
end
```

В случая опциите контролират как се генерира AST (напр. метаданни). Буквално quote означава „Не изпълнявай този код, а го представи като структура.“ Обратното на quote е unquote това означава „Вмъкни стойност вътре в кода.“

Пример 1):

```
quote do
  "Hello world!"
end
#=> "Hello world!"
```

В този пример изразът е просто низов литерал, който е представен сам по себе си в AST. Това обикновено е така за листните възли в.

Пример 2):

```
quote do
  1 + 2
end
#=> {:+, [], [1, 2]}
```

Този AST възел представлява 3-елементен кортеж :

```
+ /\
1 2
```

Това, което може да се заключи е, че нелистните AST възли са представени като три елементарни кортежи, където първият елемент е израз, идентифициращ типа на възела, а последният елемент е списък на наследниците му. Средният елемент е списък с метаданни за възела, като номера на реда и колоната.

Представяне на израза $1 + 2 * 4$ чрез AST:

```
quote do
  1 + 2 * 4
end
#=> {:+, [], [1, {:*, [], [2, 4]}]}
```

Първо, **външният кортеж** $\{:+, [], [...]\}$ е призив към $+/2$, след това наследниците са буквално 1 и призив към $*/2$, със 2 и 4 като негови наследници. Тъй като quote/2 е примитив, чрез който кодът се превръща в данни, Elixir нарича AST цитиран израз.

Терминът AST има предимството сред разработчиците, така че през повечето време и двата термина ще се използват взаимозаменящо, дори и в официалната документация на Elixir.

Едно нещо, което трябва да се отбележи тук, е, че някои литерали се изразяват сами. В много езици литералите имат свое собствено представяне, например в JavaScript литералът 1 ще бъде изразен от обекта $\{\text{type: 'Literal', value: 1}\}$ или $\{\text{integer, LineNumber, 1}\}$ кортежа в Erlang. Това не е така в Elixir и единственото обяснение за това е, че улеснява работата с тях в макроса, тъй като съвпадението по литералите е много по-малко подробно, отколкото съвпадението срещу вложените 3 кортежа. Един от недостатъците на това обаче е, че информацията за позицията или контекста на литерала се губи.

или **Пример2** може да се представи по следния начин

```
quote do
  1 + 2
end
```

Резултат

```
{:+, [context: Elixir, import: Kernel], [1,2]}
```

2.2. Какво прави AST

AST позволява:

- анализ на кода

- метапрограмиране
- макроси

Elixir компилира кода чрез AST.

Някои term-и на Elixir могат да бъдат представени сами. Например:

- atoms, като :ok
- integer, като 42
- float, като 13.1
- string, като "hello"
- list, като [1, 2, 3]
- tuples с 2 елемента, като {"hello", :world}

Забележка: Този списък е взиман от раздела Elixir AST в справочника за синтаксиса на Elixir. Това означава, че опитът да се цитират някои от тези изрази ще доведе до самия израз:

```
quote do
  {:foo, :bar}
end
#=> {:foo, :bar}
```

Причината 2-та кортежа да са специален случай е, че позволява списъците с ключови думи да бъдат изразени като литерали. Това улеснява предаването на опциите към макроси и също улеснява идентифицирането do/ end блока чрез съвпадение на шаблони [do: block].

Всеки друг вид израз е представен от 3 кортежа, който може да бъде променлив или извиквания.

2.3. Променливи

В AST променливите са представени като tuple.

Променливата ни предоставя наименувано хранилище, с което програмите ни могат да манипулират. Всяка променлива в Elixir има специфичен тип, който определя размера и разположението и в паметта; диапазона от стойности, които може да съхрани в тази памет; както и набор от операции, които могат да бъдат приложени над нея (променливата).

Elixir поддържа следните основни типове променливи:

- **Цели числа**

Те се използват за цели числа. Те са с размер 32 бита на 32-битова архитектура и 64 бита на 64-битова архитектура. Целите числа винаги са подписани в Elixir. Ако цяло число започне да се разширява по размер над границата си, Elixir го преобразува в голямо цяло число, което заема памет в диапазон от 3 до n думи, в зависимост от това, което може да се побере в паметта.

- **Числа с плаваща запетая**

Те имат 64-битова точност в Elixir. Подобни са на целите числа по отношение на паметта. При дефиниране на float може да се използва експоненциална нотация.

- **Булеви числа**

Те могат да приемат 2 стойности true, или false.

- **Низ**

Низовете са utf-8 кодирани в Elixir. Те имат модул за низове, който предоставя функционалност за манипулиране с низове.

- **Анонимни функции/лямбда**

Това са функции, които могат да бъдат дефинирани и присвоени на променлива, която след това може да се използва за извикване на тази функция.

- **Колекции**

В Elixir има много видове колекции - списъци, кортежи, map, двоични файлове и др.

Декларация на променлива

Декларацията на променлива казва на интерпретатора къде и колко да създаде хранилището за променливата. Elixir не ни позволява просто да декларираме променлива. Променливата трябва да бъде декларирана и да и се присвои стойност едновременно. Например, за да създадем променлива с име life и да й присвоим стойност 42, правим следното –

```
life = 42
```

Това ще свърже променливата със стойността 42. Ако искаме да присвоим на променливата нова стойност, можем да използваме същия синтаксис като по-горе, т.е.

```
life = "Hello world"
```

Пример

```
quote do
  x
end
```

Резултат

```
{:x, [], Elixir}
```

Наименуване на променлива

Наименуването на променливите следва конвенцията `snake_case` в Elixir, т.е. всички променливи трябва да започват с малка буква, последвана от 0 или повече букви (както главни, така и малки), последвани в края от незадължителен '?' ИЛИ '!'. Имената на променливите също могат да започват с водещо долно тире, но това трябва да се използва само при игнориране на променливата, т.е. тази променлива няма да се използва отново, а е необходимо да бъде присвоена на нещо.

Печат на променливи

В интерактивната обвивка променливите ще се отпечатват, ако просто се въведе името на променливата. Например, ако се създаде променлива :

```
life = 42
```

и се въведе „life“ в обвивката, резултатът на изхода ще е:

```
42
```

Ако се цели, извеждане на променливата в конзола (когато се изпълнява външен скрипт от файл), трябва да се предостави променлива, като вход към функцията `IO.puts`:

Пример:

```
life = 42
IO.puts life

или
life = 42
IO.puts(life)

което ще изведе на изхода:
42
```

Променливите са представени с кортеж, където първият елемент е атом, представляващ неговото име, вторият е метаданни на възела, а третият е контекста на променливата:

Пример 1:

```
quote do
  foo
end
#=> {:foo, [], Elixir}
```

Quote винаги включва модула, в който е извикан като променлив контекст. Ако този код се изпълнява в `iex`, контекстът ще бъде като в Пример 1. Ако обаче се използва `Code.string_to_quoted!` тогава, контекстът ще бъде `nil`, тъй като няма среда, в която изразът е разрешен:

Пример 2:

```
Code.string_to_quoted!("foo")
#=> {:foo, [line: 1], nil}
```

Контекстът за разрешаване може да бъде инжектиран в цитиран израз с опцията `:context` :

```
quote context: Foo do
  bar
end
#=> {:bar, [], Foo}
```


Когато компилаторът срещне макрос, той се разширява т.е., макросът се оценява и резултатът се вмъква на мястото в AST, където се е случило извикването на макроса. Това се случва рекурсивно, докато не остане макрос за разширяване. По време на този процес макросите могат да вмъкват променливи в AST и техните възли ще имат макро модула като свой контекст. Този контекст може да присъства и в context-ключовете на някои метаданни на възли.

Контекстът е полезен, за да се следи кой код кои променливи дефинира. Например, помислете за този макрос:

```
defmodule A do
  defmacro demo() do
    quote do
      foo
    end
  end
end
```

Ако го използваме в израз:

```
require A
A.demo()
```

Извикването A.demo () ще бъде разширено до:
{:foo, [counter: -576460752303422751], A}

Контекстът на променливата AST е модулет на макроса, дефинирал тази променлива, и допълнително в метаданните се вмъква брояч, който предотвратява промяната да пречи на променливите със същото име, определени извън обхвата на макроса, след като бъде оценен. Това е един от механизмите, чрез които Elixir постига хигиенни макроси: компилаторът разпознава променливите чрез name и metadata[:counter], или name и context. Макросът var!/2 премахва този брояч от променливия възел, за да отбележи, че не трябва да се хигиенизира.

2.4. Извиквания

Извикване на функция също е AST структура.

Това, което беше разгледано, бяха листните възли на синтаксичното дърво. Те представляват отделни части от данни, които не могат да бъдат намалени повече, за да се получи смислен фрагмент от синтаксис. Все още трябва да се комбинират, за да станат полезна програма и това се постигне с извиквания. Извикванията са основният градивен елемент с други възли, комбинирани за изграждане на подходящ AST. В най-основната си форма те изглеждат така:

Пример

```
quote do
  sum(1,2)
end
```

AST

```
{:sum, [], [1,2]}
```

И те са представени с кортеж с 3 елемента:

```
{:sum, [], [1, 2, 3]}
```

Тук първият елемент е атом в случай на неквалифицирани извиквания като този пример, или кортеж, представляващ извикване към .оператора в случай на квалифицирани извиквания като String.downcase("HELLO"). Вторият елемент отново е метаданни, а третият е списък с аргументи.

2.5. Оператори и конструктори

Операторите също са функции.

Пример

```
1 + 2
```

AST

```
{:+, [], [1,2]}
```

Оператори като + също са представени като неквалифицирани повиквания:

```
quote do
  1 + 2
end
#=> {:+, [], [1, 2]}
```

Основната характеристика на операторите е, че Elixir не може да анализира произволен оператор.

Структурите на данни като map, кортежи и битови низове са представени с извикване, където типът е името на атома на съответния им конструктор на специална форма, а аргументите са техните елементи. Например map:

```
%{foo: :bar}
```

Представя се от извикването:

```
{:%{}, [], [foo: :bar]}
```

Същото важи и за кортежи с размери, различни от 2:

```
quote do
  {1, 2, 3}
end
#=> {:{}, [], [1, 2, 3]}
```

или низове:

```
quote do
  <<1, 2, 3>>
end
#=> {:<<>>, [], [1, 2, 3]}
```

2.6. Псевдоними и блокове

Alias също се представя в AST.

Пример

```
alias MyApp.User
```

AST съдържа информация за модула.

Някои извиквания имат специално значение. Например имената на модулите са атоми: Enum всъщност е `:Enum` под повърхността, но те са представени в AST като `__aliases__` извикване:

```
quote do
  Foo
end
#=> {:__aliases__, [], [:Foo]}
```

Аргументите на това извикване са сегментите на име на модул:

```
quote do
  Foo.Bar.Baz
end
#=> {:__aliases__, [], [:Foo, :Bar, :Baz]}
```

Друг специален случай са кодови блокове. Блокът е последователност от два или повече израза, всеки от които е определен в различен ред. Той е представен като `__block__` извикване, където всеки ред е отделен аргумент:

```
quote do
```

```
1
2
3
end
#=> {: block , [], [1, 2, 3]}
```

Elixir също позволява да се използва точка и запетая като разделител на линии, така че този израз е еквивалентен на предишния:

```
quote do 1; 2; 3; end
#=> {: block , [], [1, 2, 3]}
```

2.7. Стрелка отляво надясно

Стрелката отляво надясно $\rightarrow$ е специален вид възел в AST (Abstract Syntax Tree) на Elixir. Тя се представя като неквалифицирано извикване и се използва за дефиниране на клаузи (patterns $\rightarrow$ expression) в конструкции като анонимни функции (fn), case, cond и receive. Този възел винаги се появява в списък от клаузи, поради което може да се използва само в контекста на такива конструкции (например в do блок или като част от анонимна функция).

Първият аргумент на възела винаги е списък, който съдържа pattern-и (аргументите или условията отляво на стрелката), а вторият аргумент е изразът, който ще бъде изпълнен, ако pattern-ът съвпадне.

Пример

```
case x do
  1 -> "one"
  2 -> "two"
end
```

В този пример всяка клауза със стрелка $\rightarrow$ съдържа pattern (1 или 2) отляво и израз отясно ("one" или "two"), който се изпълнява, когато стойността на x съвпадне със съответния pattern.

Приблизително представяне на AST за този пример:

```
{:case, meta, [
  x,
  [
    do: [
      {:->, meta, [[1], "one"]},
      {:->, meta, [[2], "two"]}
    ]
  ]
]}
```

Тук всяка клауза със стрелка се представя като възел от вида:

```
{:->, meta, [patterns, body]}
```

където patterns е списъкът с pattern-и отляво на стрелката, а body е изразът, който се изпълнява при съвпадение.

Не трябва да се бърка с pipe оператора ($|>$)

Пример

```
[1,2,3]
> Enum.map(&(&1*2))
> Enum.sum()
```

2.8. Квалифицирани извиквания

Извикване на функция от модул.

Пример

```
Enum.map([1,2,3], fn x -> x*2 end)
```

Досега бяха разгледани примери за неквалифицирани извиквания, където първият елемент е атом. Квалифицираните извиквания, от друга страна, имат извикване към .оператора като първи елемент. Този вид извикване се среща, когато има извикване от лявата страна на ()оператора (извикване). Има две такива ситуации:

Когато лявата страна е друго извикване

С .оператора

Първата ситуация възниква в случаи като foo(). Такъв синтаксис е невалиден в повечето случаи и няма да успее да се компилира, но е възможно да се използва вътре в quote, където обикновено се използва за съставяне на цитирани изрази за генериране на функции с unquote. пример:

```
quote unquote: false do
  unquote(:foo)()
end
#=> [{:unquote, [], [:foo]}, [], []]
```

Можем да се види, че първият елемент тук е неквалифицираното извикване {:unquote, [], [:foo]} вместо атом.

Втората ситуация е по-интересна и там нещата започват да стават сложни. Нека разгледаме следния синтаксис:

```
foo.bar
```

Ако го цитираме, получаваме следния AST:

```
{:., [], [{:foo, [], Elixir}, :bar]}, [no_parens: true], []]
```

Тук има доста неща за разопаковане. Първият елемент е извикване на точков оператор, където лявата страна е променлива, а дясната страна е атом:

```
{:., [], [{:foo, [], Elixir}, :bar]}
```

Външният възел е извикване без аргументи, но съдържа no_parens: true метаданни. Това предполага, че всяко използване на .оператора ще бъде представено като извикване, където скобите могат да бъдат пропуснати, а първият елемент ще бъде извикване на точков оператор.

Не е трудно също да се докаже, че извикването на точковия оператор може да бъде вложено произволно:

```
quote do
  foo.bar.baz(1, 2, 3)
end
#=>
{
  {:., [], [
    {:., [], [
      {:foo, [], Elixir}, :bar
    ]}, [], [1, 2, 3]
  ]}
}
```

Друг случай на квалифицирани извиквания е квалифициран кортежен синтаксис: Foo.{Bar, Baz}. Този синтаксис се използва най-често при псевдоним на множество модули. Той е представен в AST като {:., [], [alias, :{}]} извикване:

```
quote do
  Foo.{Bar, Baz}
end
#=>
{
  {:., [], [{:__aliases__, [], [:Foo]}, :{}]},
  [],
}
```

```
[{:__aliases__, [], [:Bar]}, {:__aliases__, [], [:Baz]}]
```

Интересните битове тук са аргументите за извикването на точки: `[alias, :{}]` аргументите са начинът, по който Elixir използва за разпознаване на квалифицирани кортежи като нещо различно от `Foo.{}(Bar, Baz)`.

Пример за цитиран израз:

```
quote do
  foo[:bar]
end
#=> [{:., [], [Access, :get]}, [], [{:foo, [], Elixir}, :bar]]
```

Това, което се случва тук, е, че синтаксисът за достъп се свежда до квалифицирано извикване до `Access.get`. Начинът, по който Elixir го разпознава в AST, е чрез `[Access, :get]` аргументите за извикване на вътрешна точка и има смисъл да има толкова аргументи, колкото `Access.get` позволява:

```
Macro.to_string({:., [], [Access, :get]}, [], [{:foo, [], Elixir}])
#=> "Access.get(foo)"

Macro.to_string({:., [], [Access, :get]}, [], [{:foo, [], Elixir}, :bar])
#=> "foo[:bar]"

Macro.to_string({:., [], [Access, :get]}, [], [{:foo, [], Elixir}, :bar, :baz])
#=> "Access.get(foo, :bar, :baz)"

Macro.to_string({:., [], [Access, :get]}, [], [{:foo, [], Elixir}, :bar, :baz, :quz])
#=> "Access.get(foo, :bar, :baz, :quz)" <-- this will fail
```

Това трябва да обхваща повечето, ако не всички, възможни конструкции, които отговарят на Elixir AST. Всеки вид AST, който откриете, ще бъде направен от композиция от споменатите по-горе възли и тяхното познаване, което ще направи четенето му и идентифицирането на моделите много по-лесно.

3. Функции от първи клас

В Elixir функциите са first-class citizens.

Това означава:

- могат да се предават като аргументи
- могат да се връщат от функции
- могат да се съхраняват в променливи

Пример:

```
f = fn x -> x * 2 end
f.(5)
```

Изход

```
10
```

В компютърното програмиране, first-class object, first-class citizen или first-class value (първокласен обект или първокласна стойност) е езиково образуване (например, функция или променлива), което работи като всеки останал обект в езика.

Например в езика за програмиране C не можем да предадем функция на друга функция като параметър. Така че в C, функциите се наричат „обекти от втори клас“. В JavaScript обаче функция може да бъде предадена на друга функция като параметър, следователно в JavaScript, функциите са първокласни.

Т.е. програмния езикът за програмиране има първокласни функции, ако третира функциите като first-class citizen. Това означава, че програмният език поддържа предаване на функции, като аргументи на други функции, връщайки ги като стойности от други функции и ги присвоява на

променливи или ги съхранява в структури от данни. [1] Някои теоретици на езиците за програмиране изискват поддръжка и за анонимни функции (функционални литерали). [2] В езици с първокласни функции имената на функциите нямат никакъв специален статус; те се третират като обикновени променливи с тип функция. [3] Терминът е въведен от Кристофър Стречи в контекста на „функциите на първокласни обекти“ в средата на 60-те години. [4]

Първокласните функции са необходимост за функционалния стил на програмиране, при който използването на функции от по-висок ред е стандартна практика. Прост пример за функция с по-висок ред е функцията на map, която приема като свои аргументи функция и списък и връща списъка, образуван чрез прилагане на функцията към всеки член от списъка. За да може езикът да поддържа map, той трябва да поддържа предаване на функция като аргумент.

Съществуват известни трудности при внедряването на функции като аргументи или връщането им като резултати, особено при наличието на нелокални променливи, въведени във вложени и анонимни функции. Исторически тези проблеми се наричат `funarg` проблеми, като името идва от "аргумент за функция". [5] В ранните императивни езици тези проблеми са избегнати чрез или неподдръжане на функции като типове резултати (напр. ALGOL 60, Pascal) или пропускане на вложени функции и по този начин нелокални променливи (напр. C). Ранният функционален език Lisp възприел подхода на динамичното обхващане, където нелокалните променливи се отнасят до най-близкото определение на тази променлива в точката, където се изпълнява функцията, вместо където е дефинирана. Подходящата поддръжка за първокласни функции с лексикален обхват е въведена в Scheme и изисква справки за работа да функционират като затваряне вместо празни указатели на функции, [4] което от своя страна прави събирането на боклук необходимост.

Според "Elixir in Action", в Elixir функцията е първокласен гражданин, което означава, че може да бъде присвоена на променлива. Тук присвояването на функция на променлива не означава извикване на функцията и съхранение на резултата и в променлива. Вместо това се присвоява самата дефиниция на функцията и може да се използва променливата за извикване на тази функция. Например:

За да създадете функционална променлива, можете да използвате `fn` конструкцията:

```
iex(1)> square = fn x ->
  x * x
end
```

Променливата квадрат сега съдържа функция, която изчислява квадрата на число. Тъй като функцията не е обвързана с глобално име, тя се нарича `anonymous function` или `lambda` (анонимна функция или ламбда).

Забележете, че списъкът с аргументи не е затворен в скоби. Технически можем да използваме скоби тук, но преобладаващата конвенция, също наложена от форматиращия Elixir, е да се пропуснат скобите. За разлика от това, списък с аргументи на наименувана функция трябва да бъдат оградени в скоби. На пръв поглед това изглежда несъответстващо, но има добра причина за тази конвенция, тя ще се разгледа по-късно. За да извикаме тази функция, посочваме името на променливата, последвано от точка (.) и аргументите:

```
iex(2)> square.(5)
25
```

ЗАБЕЛЕЖКА: Може да се чудите защо тук е необходим точков оператор. Мотивацията зад оператора точка е да се направи кода по-ясен. Когато в изходния код се среща `square.(5)` израз, трябва да се знае, че се извиква анонимна функцията. Обратно, изразът `square (5)` извиква наименува функция, определена някъде другаде в модула. Без оператора точка ще трябва да анализираме околния код, за да разберем дали сме извикали наименувана или анонимна функция. Тъй като функциите могат да се съхраняват в променлива, те могат да бъдат предадени като аргументи на други функции. Това често се използва, за да се позволи на потребителите да параметризират общата логика. Например функцията `Enum.each/2` реализира общата итерация - може да повтори всичко изброимо, например списъци. Функцията `Enum.each/2` приема два аргумента: `enumerable` и `lambda` с едно `arity` (`arity` - т.е. анонимна функция, която приема един аргумент). Той се повтаря през `enumerable` и извиква `lambda` за всеки елемент. Потребителите предоставят `lambda`, за да посочат какво искат да правят с всеки елемент.

Следният фрагмент използва `Enum.each` за отпечатване на всяка стойност на списък на екрана:

```

iex(3)> print_element = fn x -> IO.puts(x) end  ← Defines the lambda
iex(4)> Enum.each(
  [1, 2, 3],
  print_element  ← Passes the lambda to Enum.each
)

1 |
2 |   Output printed by the lambda
3 |
:ok  ← Return value of Enum.each

```

Разбира се, нямате нужда от временна променлива, за да предадете lambda на Enum.each:

```

iex(5)> Enum.each(
  [1, 2, 3],
  fn x -> IO.puts(x) end  ← Passes the lambda directly
)

1
2
3

```

Забележете как lambda просто препраща всички аргументи към IO.puts, без да върши друга значима работа. За такива случаи Elixir дава възможност директно да се позовава на функцията и имат по-компактна lambda дефиниция.

Вместо да се пише `fn x -> IO.puts(x) end`, може да се напише `&IO.puts/1`.

Операторът `&`, известен също като capture оператор (или оператор на улавяне), приема пълния квалификатор на функцията - име на модул, име на функция и arity - и превръща тази функция в lambda, която може да бъде присвоена на променлива. Може да използвате оператора за улавяне, за да опростите извикването към Enum.each:

```

iex(6)> Enum.each(
  [1, 2, 3],
  &IO.puts/1  ← Passes the lambda that delegates to IO.puts
)

```

Операторът за улавяне може също да се използва за съкращаване на дефиницията на lambda, правейки я с възможност да се пропусне изрично наименоуване на аргументи. Например, можем да обърнем това определение:

```

iex(7)> lambda = fn x, y, z -> x * y + z end
в по-компактна форма:
iex(8)> lambda = &(&1 * &2 + &3)

```

Този фрагмент създава lambda с три arity. Всеки аргумент се препраща чрез `& n` заместител, който идентифицира n-тия аргумент на функцията. Можете да наречете този lambda като всеки друг:

```

iex(9)> lambda.(2, 3, 4) #т.е. 2*3+4=10
10

```

Върнатата стойност $10 = 2 * 3 + 4$, както е посочено в дефиницията на lambda.

3.1. Closures

Closure е функция, която запомня средата си.

Пример

```

x = 10
f = fn y -> x + y end
f.(5)

```

Изход

```
15
```

Функцията използва x от външния scope.

lambda може да се позовава на всяка променлива от външния обхват:

```
iex(1)> outside_var = 5
5

iex(2)> my_lambda = fn ->
  IO.puts(outside_var)
end

iex(3)> my_lambda.()
5
```

← Lambda references a variable from the outside scope

Докато държим препратката към `my_lambda`, променливата `outside_var` също е достъпна. Това също е известно като Closures: като държим препратка към ламбда, косвено се съдържа препратка към всички променливи, които използва, дори ако тези променливи са от външен обхват.

Closures винаги улавя определено място в паметта. Превързването на променлива не засяга предварително дефинираната ламбда, която препраща към същото символично име:

```
iex(1)> outside_var = 5
iex(2)> lambda = fn -> IO.puts(outside_var) end
iex(3)> outside_var = 6
iex(4)> lambda.()
5
```

← Lambda captures the current location of outside_var

← Rebinding doesn't affect the closure.

Proof that the closure isn't affected

Предидният код илюстрира друг важен момент. Обикновено, след като отскочи `outside_var` до стойността 6, оригиналното място на паметта би било допустимо за сметосъбиране. Но тъй като ламбда функцията улавя оригиналното местоположение (този, който държи номер 5) и все още споменаваме тази ламбда, първоначалното местоположение не е достъпно за събирача на боклук.

Нека да припомним какво е анонимната функция – такава, която няма име. Както видяхме при Enum, те често се предават на други функции. За да се дефинира анонимна функция в Elixir, се нуждаем от ключовите думи `fn` и `end`. В тях можем да определим произволен брой параметри и функционални тела, разделени с `->`. Нека разгледаме един основен пример:

```
iex> sum = fn (a, b) -> a + b end iex> sum.(2, 3) #5
```

3.2. Стенографията &

Elixir има кратък синтаксис за анонимни функции.

Пример

```
Enum.map([1,2,3], &(&1 * 2))
```

В случая `&1` означава първи аргумент.

Използването на анонимни функции е толкова обичайна практика в Elixir, че има стенография за това:

```
iex> sum = &(&1 + &2)
iex> sum.(2, 3)
5
```

Както вероятно вече се досещате, в стенографската версия нашите параметри са ни достъпни като `&1`, `&2`, и `&3` и т.н.

Анонимните функции са първокласни стойности, така че те могат да бъдат предадени като аргументи на други функции, а също така могат да връщат стойност. Съществува специален синтаксис, който позволява да се третират наименуваните функции по същия начин.

```
defmodule Math do
  def square(x) do
    x * x
  end
end
```



```
end
Enum.map [1, 2, 3], &Math.square/1
#=> [1, 4, 9]
```

4. Анонимни функции

Анонимните функции се дефинират с `fn`.

Синтаксис

```
fn arguments -> expression end
```

Пример

```
add = fn a, b ->
  a + b
end

add.(2,3)
```

Изход

```
5
```

Анонимна функция с pattern matching

```
f = fn
  {:ok, x} -> x
  {:error, _} -> 0
end

f.({:ok, 10})
```

Изход

```
10
```

Анонимните функции се дефинират по следния начин:

```
sum = fn(a, b) -> a + b end
sum.(4, 3)
#=> 7

square = fn(x) -> x * x end
Enum.map [1, 2, 3, 4], square
#=> [1, 4, 9, 16]
```

Възможно е да се използва и съвпадение на шаблони при определяне на анонимни функции

```
f = fn
  {:a, :b} = tuple ->
    IO.puts "All your #{inspect tuple} are belong to us"
  [] ->
    "Empty"
end

f.([])
#=> "Empty"

f.({:a, :b})
#=> "All your {:a, :b} are belong to us"
```

Анонимните функции са разделени от ключовите думи `fn` и `end`:

```
iex> add = fn a, b -> a + b end
#Function<12.71889879/2 in :erl_eval.expr/5>
iex> is_function(add)
true
iex> is_function(add, 2)
```

```
true
iex> is_function(add, 1)
false
iex> add.(1, 2)
3
```

Функциите са „първокласни функции“ в Elixir, което означава, че могат да бъдат предадени като аргументи на други функции, точно както целите числа и низовете. В примера сме предали функцията в променливата `add` към функцията `is_function/1`, която връща `true`. Можем също така да проверим достоверността на функцията, като извикаме `is_function/2`.

Обърнете внимание, че точка `(.)` между променливата и скобите е необходима за извикване на анонимна функция.

Анонимните функции са затворени и като такива те имат достъп до променливи, които са в обхвата, когато функцията е дефинирана:

```
iex> add_two = fn a -> add.(a, 2) end
#Function<6.71889879/1 in :erl_eval.expr/5>

iex> add_two.(2)
4
```

Имайте предвид, че променлива, присвоена във функцията, не влияе на заобикалящата я среда:

```
iex> add_two = fn a -> add.(a, 2) end
#Function<6.71889879/1 in :erl_eval.expr/5>

iex> add_two.(2)
4
```

Много функции в модула `Enum` в Elixir приемат анонимни функции като аргумент за работа с всяка итерация на преброянето, което се предава.

Тези анонимни функции често се пишат стенографски с помощта на оператора `Capture (&)`.

Ето няколко примера, които показват как операторът за улавяне може да бъде реализиран с модула `Enum`. Всяка версия е функционално еквивалентна.

Използване на оператора за улавяне с анонимна функция

По-долу е типичен пример за стандартния синтаксис при предаване на анонимна функция на `Enum.map/2`.

```
iex> Enum.map([1,2,3], fn number -> number + 3 end)
[4, 5, 6]
```

Сега прилагаме оператора за улавяне `(&)`; улавяне на всяка итерация от списъка с числа `[1,2,3]` и присвояване на всяка итерация към променливата `&1`, докато се предава през функцията за картографиране.

```
iex> Enum.map([1,2,3], &(&1 + 3))
[4, 5, 6]
```

Това може да бъде допълнително преработено, за да се присвои предишната анонимна функция с оператора `Capture` на променлива и да се извика от функцията `Enum.map/2`.

```
iex> plus_three = &(&1 + 3)
iex> Enum.map([1,2,3], plus_three)
[4, 5, 6]
```

Използване на оператора за улавяне с наименувана функция

Първо създаваме наименувана функция и я извикваме в рамките на анонимната функция, определена в `Enum.map/2`.

```
defmodule Adding do
  def plus_three(number), do: number + 3
end

iex> Enum.map([1,2,3], fn number -> Adding.plus_three(number) end)
[4, 5, 6]
```

След това можем да я преработим, за да използваме оператора `Capture`.

```
ix> Enum.map([1,2,3], &Adding.plus_three(&1))  
[4, 5, 6]
```

За най-краткия синтаксис можем директно да извикаме наименуваната функция без изрично улавяне на променливата.

```
ix> Enum.map([1,2,3], &Adding.plus_three/1)  
[4, 5, 6]
```

Обобщение

- Elixir има динамична типова система
- съществуват примитивни и комбинирани типове
- кодът се представя чрез AST
- функциите са `first-class`
- широко се използват анонимни функции
- shorthand синтаксисът `&` улеснява писането на функции.